Course 50.050/50.550
Advanced Algorithms

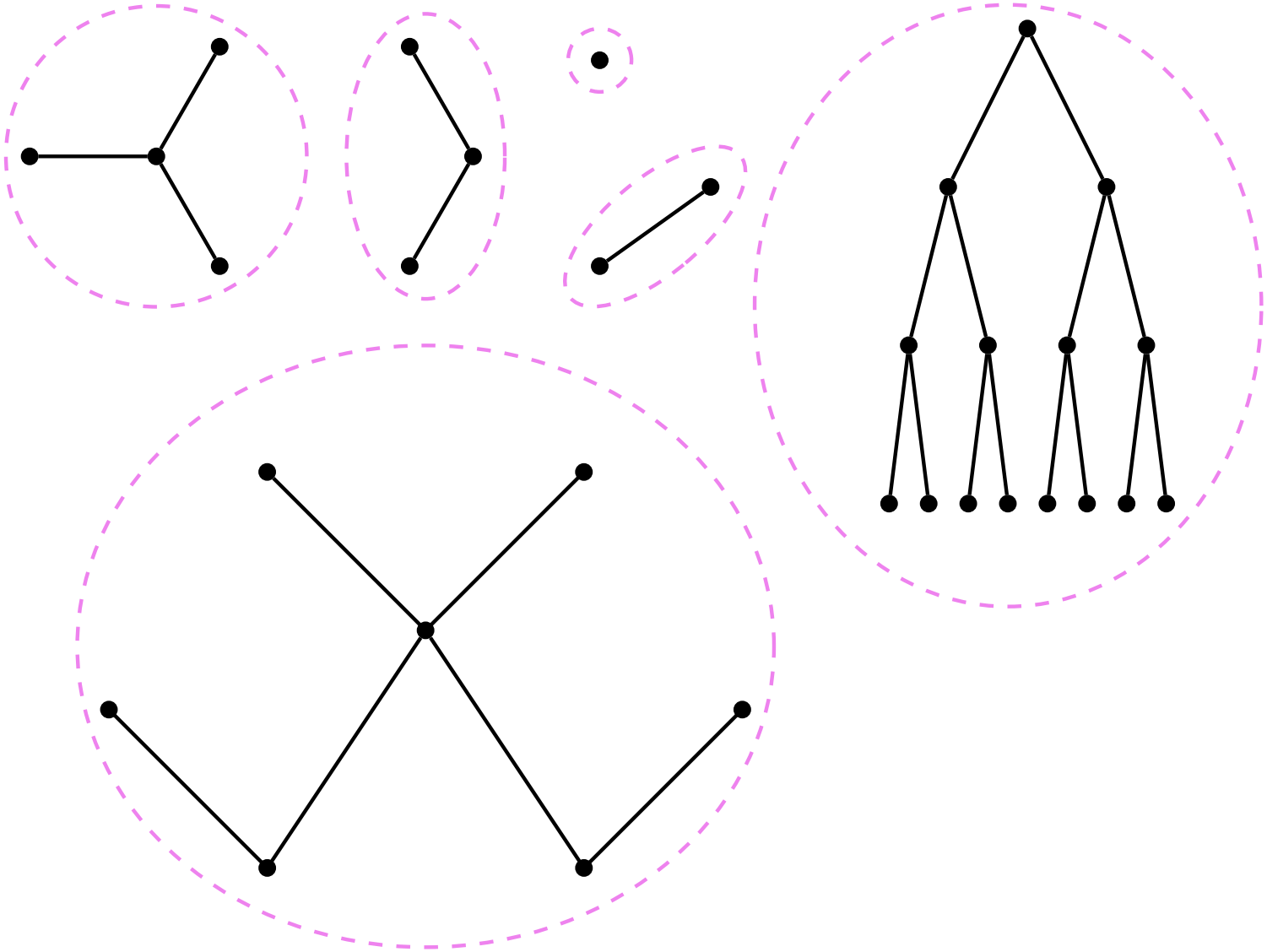
Week 6 – Lecture L06.01

SINGAPORE UNIVERSITY OF
TECHNOLOGY AND DESIGN

Outline of Lecture

- Trees, forests, rooted trees, DAGs
- Spanning trees
- Vertex labeling, edge labeling, weighted graphs
- Minimum spanning trees, cuts, cut-sets
- Intuition for computing minimum spanning trees

Informal intuition for trees and forests



- An **undirected graph** with 6 connected components.
- A **forest** with 6 trees.

What is a tree? What is a forest?

Definition: Let $G = (V, E, \phi)$ be a graph.

- If G is **undirected**, then:
  - G is a tree if $|V| \geq 1$, G is connected, and G has no closed trails with at least one edge.
    - **Recall:** A closed trail is a walk $(v_0, e_1, v_1, \ldots, v_{k-1}, e_k, v_k)$ **with** distinct edges, such that $v_0 = v_k$.
  - G is a forest if every connected component of G is a tree.
- If G is **directed**, then:
  - G is a tree (or directed tree or polytree) if the underlying undirected graph of G is a tree;
  - G is a forest (or directed forest or polyforest) if the underlying undirected graph of G is a forest.

Warning: Many authors insist that a "tree" is always **undirected**, and so prefer the terminology "polytree" for the directed case, in which case a "polytree" is technically not a tree, since it isn't undirected.

- Analogously, many authors insist that a "forest" is always **undirected**, and so prefer to use "polyforest" for the directed case.

50.050 Convention: A tree/forest can be undirected or directed.

- We will be explicit if we mean only one of these two cases specifically.
- Otherwise, when we say that T is a tree/forest, it could be either undirected or directed.

Convention for trees and forests

Notation: A tree is commonly denoted by T, rather than G.

- A vertex x in a tree satisfying $\deg(x) = 1$ is called a leaf.

Note: K_1 is an (undirected) tree. A null graph is an (undirected) forest.

- **Recall:** A null graph is an undirected graph whose edge set is empty, and contains (possibly many) isolated vertices.
- **Note:** By convention, K_0 (null graph with 0 vertices) is considered to be a forest with 0 trees. For this to be true, K_0 cannot be a tree.

Trees and forests (whether undirected or directed) are **simple** graphs.

- So we could write "Let $T = (V, E)$ be an (undirected) tree .."
- A subtree of a tree T is a subgraph of T that is a tree.
  - In general, a subgraph of a tree is a forest.
- A subforest of a forest G is a subgraph of G.
  - **Note:** If G is a graph has no closed trails with least one edge, then every subgraph of G also has no closed trails with at least one edge.
  - Thus, a subgraph of a forest is always a forest.

Warning: Many authors define "tree" as "connected acyclic graph", and define a graph to be "acyclic" to mean "having no cycles". This definition is perfectly fine with a suitable definition of "cycle", but not all authors are careful!

Basic properties of trees and forests

Theorem: Let T be an undirected tree. Then for any two vertices u, v of T, there exists a unique trail from u to v.

Proof: Assume that $u \neq v$, and suppose instead there are two distinct trails $(v_0, e_1, v_1, \ldots, v_k)$, $(v'_0, e'_1, v'_1, \ldots, v'_\ell)$ from u to v. Note that $v_0 = v'_0 = u$, and $v_k = v'_\ell = v$. Thus, $(v_0, e_1, v_1, \ldots, v_k, e'_{\ell-1}, \ldots, e'_1, v'_0)$ is a closed walk that contains a closed trail with ≥ 1 edge, i.e. a contradiction. $\square$

Corollary: Every edge of an undirected tree T is a bridge.

Proof: Suppose instead there is a non-bridge e in T, and let u, v be the endpoints of e. Since $T \backslash e$ is connected, there is a walk from u to v in $T \backslash e$, which when augmented by e and u gives a closed walk from u to u in T, which contains a closed trail with edge e, i.e. a contradiction. $\square$

Theorem: Every undirected tree T with ≥ 2 vertices has at least two leaves. *(Recall: A leaf in a tree is a vertex x satisfying $\deg(x) = 1$.)*

Proof: Let $\mathbf{p} = (v_0, e_1, v_1, \ldots, v_k)$ be a trail in T with the largest possible length. Neither v_0 nor v_k can be adjacent to any vertex not in $\mathbf{p}$, otherwise we would get a trail with a larger length. Note that v_0 is adjacent to v_1 but not to any other vertex in $\mathbf{p}$, otherwise T would have a closed trail. Hence, $\deg(v_0) = 1$. A similar argument yields $\deg(v_k) = 1$. $\square$

Basic properties of trees and forests (continued)

Theorem: An undirected tree T is bipartite.

Proof: Fix a vertex x in T. For any $y \in V(T)$, let $d(x, y)$ be the distance from x to y in T. Then T is bipartite with bipartition (V_1, V_2), where $V_1 := \{y \in V(T) | d(x, y) \text{ is odd}\}$, and $V_2 := \{y \in V(T) | d(x, y) \text{ is even}\}$. $\square$

Corollary: Every (undirected or directed) forest is bipartite.

Theorem: An undirected tree T with n vertices has $n - 1$ edges.

Proof: (By induction on n.) Base case $n = 1$ (i.e. $T = K_1$) is true. Induction step ($n \geq 2$): Previous theorem says there is a leaf x in T. Then $T - x$ has $n - 1$ vertices and one less edge than T. By induction hypothesis, $T - x$ has $n - 2$ edges. Thus T has $n - 1$ edges. $\square$

Corollary: Every undirected forest with n vertices and m connected components has $n - m$ edges.

Proof Idea: By induction on number of connected components.

Important Theorem: Let $n, m \in \mathbb{Z}^+$. An undirected graph with n vertices, $n - m$ edges, and m connected components, is a forest.

- Can you prove this? You have all the ideas to prove this.

Corollary: Let $n \in \mathbb{Z}^+$. A connected undirected graph with n vertices and $n - 1$ edges must be a tree.

Rooted trees

Definition: An (undirected or directed) tree T is called a rooted tree if T has a distinguished vertex, which we call the root.

- What does "distinguished vertex" mean? It just means that among all the vertices in $V(T)$, there is one particular vertex that we have selected.
  - **Intuition:** We are treating this "specially chosen" vertex differently from the remaining vertices. This vertex is called the root.
    - All other vertices are called non-roots.
  - Once the root has been "singled out", we can consider the **unique** simple path from the root to any particular vertex in the tree T.
    - **Recall:** A simple path is a sequence of edges "corresponding to" a walk with distinct vertices.

Definition: Let T be a rooted tree with root r, let $x, y \in V(T)$, and let $\mathbf{p} = \langle v_0, v_1, \ldots, v_{k-1}, v_k \rangle$ be the unique simple path from r to x.

- $\left(\begin{smallmatrix} y \text{ is an} \\ \text{ancestor of } x \end{smallmatrix}\right) \Leftrightarrow \left(\begin{smallmatrix} x \text{ is a} \\ \text{descendant of } y \end{smallmatrix}\right) \Leftrightarrow \left(\begin{smallmatrix} x \neq y \text{ and} \\ y \text{ is a vertex in } \mathbf{p} \end{smallmatrix}\right)$
- $\left(\begin{smallmatrix} y \text{ is the} \\ \text{parent of } x \end{smallmatrix}\right) \Leftrightarrow \left(\begin{smallmatrix} x \text{ is a} \\ \text{child of } y \end{smallmatrix}\right) \Leftrightarrow (y = v_{k-1})$
- $\left(\begin{smallmatrix} y \text{ is a} \\ \text{sibling of } x \end{smallmatrix}\right) \Leftrightarrow \left(\begin{smallmatrix} y \text{ is a} \\ \text{sibling of } x \end{smallmatrix}\right) \Leftrightarrow \left(\begin{smallmatrix} x \text{ and } y \text{ have} \\ \text{the same parent} \end{smallmatrix}\right)$.
- The subtree of T rooted at x is the rooted tree T' with root x, such that T' is the vertex-induced subgraph of T induced by $\{x\} \cup \{\text{all descendants of } x\}$.

What are acyclic graphs?

Warning: Remember the mess we discussed for the definitions of paths and cycles? The mess extends to the notion of "acyclic".

- There are two common (but **different**) definitions for "acyclic", both of which mean "having no cycles", based on different definitions for "cycles".
  - Some define "acyclic" to be what call "having no closed trails with at least one edge".
    - Seems to be more commonly used among mathematicians.
  - Some define "acyclic" to be what we call "having no simple cycles".
    - Seems to be more commonly used among computer scientists.
- **Example:** Consider the undirected graph K_2 depicted by $u \overset{e}{\bullet\!\!-\!\!\!-\!\!\bullet} v$
  - K_2 is acyclic for first definition.
  - K_2 not acyclic for second definition; it has the simple cycle (u, e, v, e, u).
  - **Consequence:** The second definition is "useless" for undirected graphs and does not match our intuition for "no cycles", since it is equivalent to "having no edges". This second definition is used in [CLRS].
- Fortunately, for **directed graphs**, both definitions of "acyclic" coincide.
  - **Note:** Be careful! A closed trail with at least one edge is not necessarily a simple cycle. (Hint: Try "extending" a simple cycle by a new loop.)
- **Convention for 50.050:** We shall use the first definition for "acyclic".

Definition: A graph G is called acyclic if G does not have any closed trail with at least one edge.

- **Consequence:** An **undirected** graph G is a tree if G is acyclic and connected, and $V(G) \neq \emptyset$.

Directed acyclic graphs

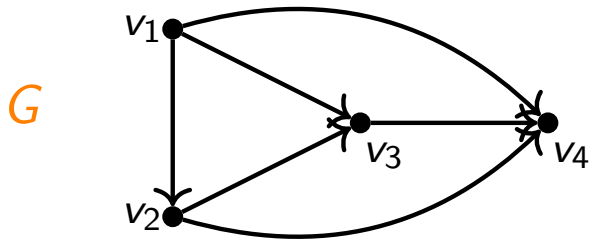
Definition: A DAG (or dag) is a directed acyclic graph.

- **Recall:** A graph G is called acyclic if G does not have any closed trail with at least one edge.
  - **Recall:** A closed trail in G is a walk $(v_0, e_1, v_1, \ldots, e_k, v_k)$ from v_0 to $v_k = v_0$, such that the edges in the walk are distinct.
    - In particular, each (v_{i-1}, v_i) must be a directed edge in $E(G)$.

Note: A DAG is NOT necessarily a directed forest!

- Similarly, a connected DAG is NOT necessarily a directed tree!

Example: A connected DAG that is not a directed tree.



Note: However, directed trees and directed forests are DAGs.

Recall: In 50.004, we learned to use DFS to topologically sort DAGs.

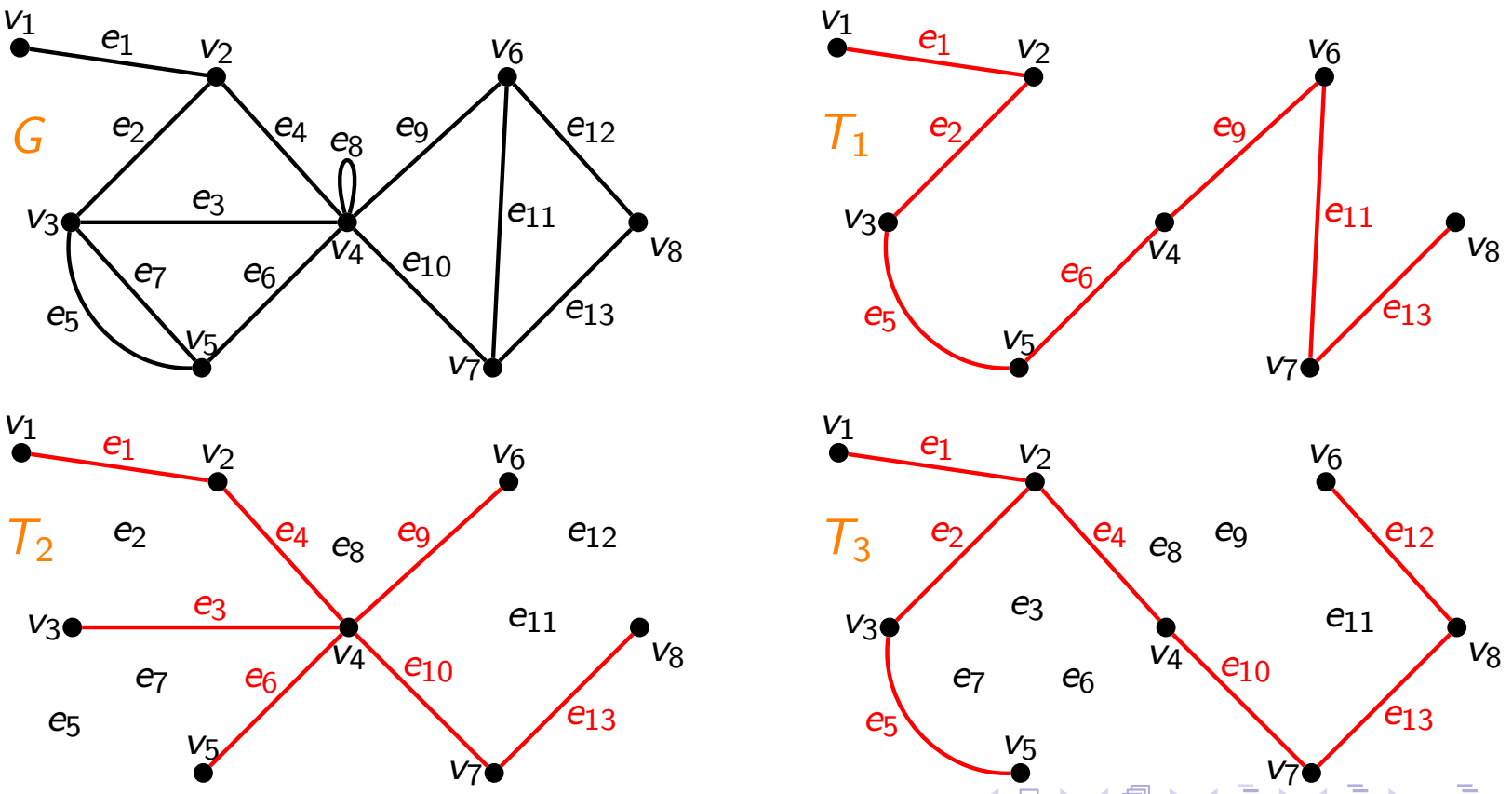
- So DFS can be used to topologically sort directed trees and directed forests.

Spanning tree

Definition: Let G be an **undirected** graph. A spanning tree of G is a subgraph T of G containing all vertices of G, such that T is a tree.

- T need not be a vertex-induced or edge-induced subgraph.
- G could have multiple spanning trees.

Example: A graph G with three different spanning trees T_1, T_2, T_3.



Basic properties of spanning trees

Theorem: Let G be an **undirected** graph. Then G has a spanning tree if and only if G is connected.

Proof: [Part 1:] G has a spanning tree T implies G is connected.

- T contains all vertices of G, so for any two vertices u, v of T, there is a (unique) trail from u to v in T, and hence in G as well.

[Part 2:] G is connected implies G has a spanning tree.

- **Run DFS on G**, and let T' be resulting the DFS forest of G.
  - G is connected, so T' is in fact a directed tree, so the underlying undirected graph T of T' is a spanning tree of G. □

Note: Alternatively, we could run breadth-first search (BFS) on G. Then the underlying undirected graph of the BFS tree of G is a spanning tree of G.

Theorem: Let G be a **connected undirected** graph, and let T be a spanning tree of G. Then T contains all bridges of G.

Proof: Suppose G has a bridge e such that e is not an edge of T.

- Then T is a subgraph of $G \backslash e$ containing all vertices of G.
  - Thus by definition, T is a spanning tree of $G \backslash e$.
- However, e is a bridge, so $G \backslash e$ is disconnected, which then contradicts the previous theorem. □

Graph labelings

Idea: For graphs, we can label their vertices, and label their edges.

- These labels are not necessarily the "names"; we could have multiple labels, and the labels could repeat.

Let $G = (V, E, \phi)$ be a graph, and let L be a non-empty set, which we shall call the **label set**, and whose elements we sall call "**labels**".

- A vertex labeling of G (w.r.t. L) is a function $\varphi : V \to L$.
- An edge labeling of G (w.r.t. L) is a function $\varphi : E \to L$.

Example: A **vertex coloring** is a special case of vertex labeling φ, defined for those graphs G that are undirected with no loops, such that for every $e \in E$, the pair $\phi(e) = \{u, v\}$ satisfies $\varphi(u) \neq \varphi(v)$.

- Similarly, an **edge coloring** is a special case of edge labeling.

Example: A **Sperner coloring** (from L05.03) is a different special case of vertex labeling φ that is NOT a vertex coloring.

Definition: If the label set L is numerical, i.e. $L \subseteq \mathbb{C} \cup \{\pm\infty\}$, then an **edge labeling** $\varphi : E \to L$ is called a weight function of G.

- The values $\phi(e)$ (for $e \in E$) are called weights.

Weighted graphs

Idea: A **weighted graph** is a graph together with one or more weight functions.

- **Note:** Most sources use this idea as the definition, without explaining what "together with" means.
  - There is no standardization for defining "weighted graph" formally.

Definition 1: Let $k \in \mathbb{Z}^+$. A weighted graph with k weight functions is a $(k+1)$-tuple $(G, w_1, \ldots, w_k)$, where G is a graph, and $w_i : E(G) \to \mathbb{R}$ is the i-th weight function of G (for $1 \leq i \leq k$).

- Hence, a weighted graph could look like $((V, E, \phi), w_1, w_2, w_3)$.
  - We distinguish the "unweighted" graph G from the weight functions.

Definition 2: Let $k \in \mathbb{Z}^+$. A weighted graph with k weight functions is a $(k+3)$-tuple $G = (V, E, \phi, w_1, \ldots, w_k)$, where V is the vertex set of G, E is the edge set of G, ϕ is the incidence function of G, and each $w_i : E(G) \to \mathbb{R}$ is the i-th weight function of G (for $1 \leq i \leq k$).

- Hence, a weighted graph could look like $(V, E, \phi, w_1, w_2, w_3)$.
  - We treat the weight functions as "additional" attributes of G.

Guideline: Choose one of the definitions, and be consistent.

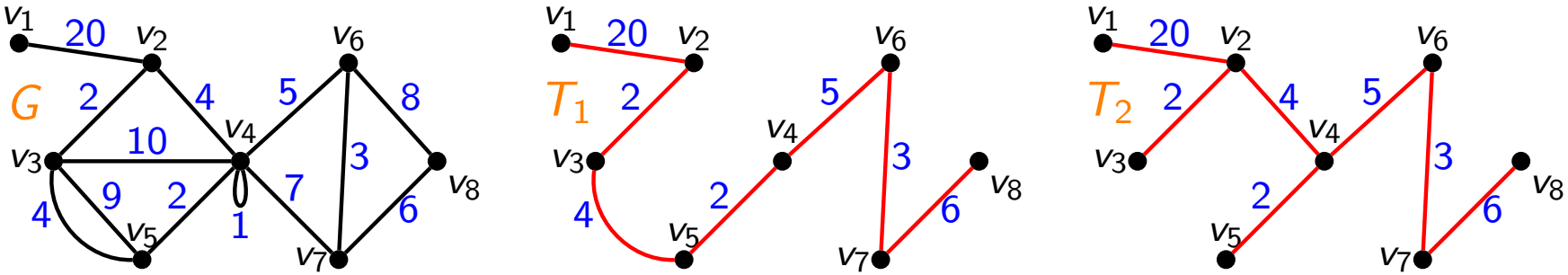
- When presenting graph algorithms on weighted graphs, be clear about which definition you use.

Minimum spanning trees

Definition: Let (G, w) be a weighted graph, where $G = (V, E, \phi)$ is a **connected undirected** graph, and $w : E \to \mathbb{R}$ is a weight function for G. Suppose $T = (V, E', \phi')$ is a spanning tree of G.

- The weight of T (w.r.t. w), denoted by $w(T)$, is the sum of the weights of edges in T, i.e. $w(T) := \sum_{e \in E'} w(e)$.
  - **Notation:** If the weight function has a different name, e.g. $\alpha : E \to \mathbb{R}$, then the weight of T (w.r.t. α) is denoted by $\alpha(T)$.
- We say that T is a minimum spanning tree of (G, w) if T has the minimum weight (w.r.t. w) over all possible spanning trees of G.
  - **Note:** Although there is only one minimum weight, there could be **multiple** minimum spanning trees with this minimum weight.

Example: Two minimum spanning trees T_1, T_2 of a graph G.



- Weights are indicated in blue.

Deletion of multiple edges and multiple vertices

Definition: Let $G = (V, E, \phi)$ be a graph, and let $D \subseteq E$ be a subset of the edges. The deletion of D from G, denoted by $G \backslash D$ or $G - D$, is the graph $G' = (V', E', \phi')$ where $V' = V$, $E' = E \backslash D$, and where the incidence function ϕ' is the restriction of ϕ to domain E'.

Definition: Let $G = (V, E, \phi)$ be a graph, and let $U \subseteq V$ be a subset of the vertices. The deletion of U from G, denoted by $G - U$ (or $G \backslash U$), is the graph $G' = (V', E', \phi')$ where $V' = V \backslash U$,

$$E' = E \backslash \{\text{all edges incident to any vertex in } U\},$$

and the incidence function ϕ' is the restriction of ϕ to domain E'.

function DELETEEDGES(G, D)
  Require: $G = (G.V, G.E, G.phi)$ is a graph.
  Require: $G.Inc$ is the incidence list of G.
  Require: D is an array of edges in $G.E$.
1: **for** each edge e in D **do**
2: DELETEEDGE(G, e)

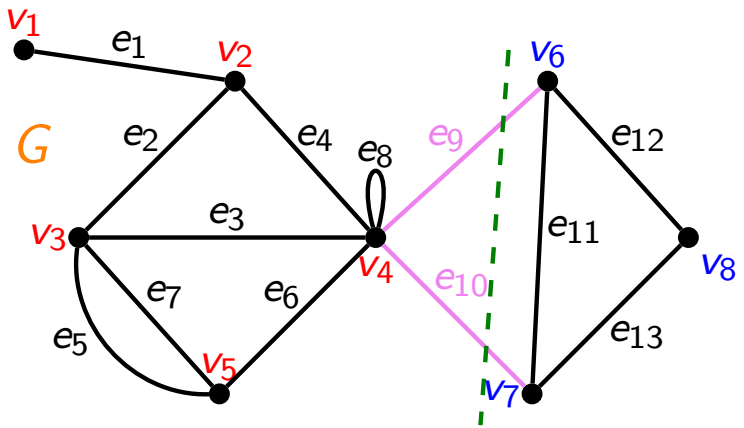
function DELETEVERTICES(G, U)
  Require: $G = (G.V, G.E, G.phi)$ is a graph.
  Require: $G.Inc$ is the incidence list of G.
  Require: U is an array of vertices in $G.V$.
1: **for** each vertex u in U **do**
2: DELETEVERTEX(G, u)

Cuts and cut-sets

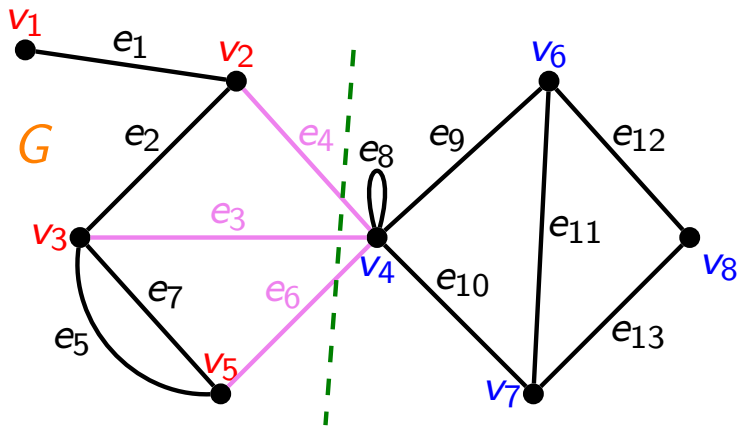
Definition: Let $G = (V, E, \phi)$ be a graph with at least 2 vertices.

- A cut of G is an ordered pair $C = (V_1, V_2)$ that is a partition of V.
  - *i.e. V_1 and V_2 are disjoint non-empty subsets whose union is V.*
  - (If G is undirected): Given an edge $e \in E$, we say that e crosses the cut C if e has one endpoint in V_1, and one endpoint in V_2.
  - (If G is directed): Given an edge $e \in E$, we say that e crosses the cut C if $\phi(e) = (u, v)$ for some $u \in V_1$, $v \in V_2$.
- The cut-set of a cut C is the set of edges in E that crosses C.

Example: Same graph G with two different cuts and cut-sets.



cut $C = (\{v_1, v_2, v_3, v_4, v_5\}, \{v_6, v_7, v_8\})$
cut-set $= \{e_9, e_{10}\}$



cut $C' = (\{v_1, v_2, v_3, v_5\}, \{v_4, v_6, v_7, v_8\})$
cut-set $= \{e_3, e_4, e_6\}$

Some properties of cuts and cut-sets

Let $G = (V, E, \phi)$ be a **connected undirected** graph, where $|V| \geq 2$.
Let $C = (V_1, V_2)$ be a cut of G, and let E_C denote the cut-set of C.

- **Recall:** A cut of G is an ordered pair that partitions V.
- **Recall:** The cut-set of C is the set of all edges in E that crosses cut C.

Theorem: Let u, v be vertices such that $u \in V_1$, $v \in V_2$. Then any walk from u to v must contain at least one edge in the cut-set of C.

Proof Idea: Any walk from u to v must have an edge that crosses C.

Corollary: If E_C has only one edge e, then e must be a bridge of G.

Theorem: The deletion $G \backslash E_C$ is disconnected.

Proof: Let u, v be vertices such that $u \in V_1$, $v \in V_2$.

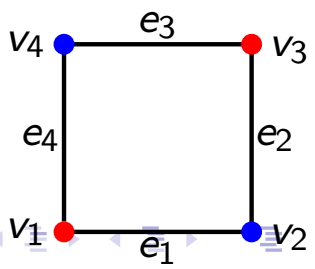
- By previous theorem, there is no walk from u to v in $G \backslash E_C$.
  - Thus, $G \backslash E_C$ is disconnected. $\square$

Note: $G \backslash E_C$ may have > 2 connected components.

- $G \backslash E'$ may still be disconnected for a proper subset $E' \subsetneq E_C$.

Example: Let G be the graph depicted on the right.

- Consider the cut $C = (\{v_1, v_3\}, \{v_2, v_4\})$.
- The cut-set of C is $E_C = \{e_1, e_2, e_3, e_4\}$.
- So $G \backslash E_C$ is a null graph with 4 vertices.



Computing minimum spanning trees

Let (G, w) be a weighted graph, where $G = (V, E, \phi)$ is **connected**, undirected, and has n vertices, and $w : E \to \mathbb{R}$ is a weight function of G.

General strategy for computing minimum spanning trees:

- Initialize F as the null graph with n isolated vertices.
  - F is initially a forest with n trees.
  - Every minimum spanning tree of (G, w) must contain all vertices, and so must have F as a subgraph.
- Iteratively insert edges into F so that the new graph F satisfies the following **invariants**:
  - F always remains as a forest.
  - F is always a subgraph of some minimum spanning tree of (G, w).

Key Idea: In each iteration, the insertion of a new edge would "combine" two trees into one "bigger" tree, so that the number of trees in the forest decreases by 1.

- After $n - 1$ iterations, our forest F becomes a tree.
- From our second invariant, our final tree F is a subgraph of some minimum spanning tree of (G, w).
  - F has $n - 1$ edges, and every spanning tree of G must have $n - 1$ edges. Hence, F itself is a minimum spanning tree!

Computing minimum spanning trees (continued)

Let $G = (V, E, \phi)$ be a **connected** undirected graph with n vertices, and let (G, w) be a weighted graph with weight function $w : E \to \mathbb{R}$.

- We shall introduce "auxiliary" definitions to help us better conceptualize things.

Definition: (**For 50.050 only**) Suppose $F = (V, E', \phi')$ is a subgraph of G that satisfies our **two invariants** (from the previous slide).

(i.e. F is a forest, and F is a subgraph of a minimum spanning tree of (G, w).)

- If $e \in E$ is not an edge of F, then we say that e is a safe edge for F (relative to (G, w)) if the new subgraph obtained after the insertion of edge e into F still satisfies our **two invariants**.
  - **Intuition:** If we treat this computation as a "game", then the safe edges are the possible "moves" we can make.

Definition: (**For 50.050 only**) Let $C = (V_1, V_2)$ be a **cut** of G, let $E' \subseteq E$ be any subset of edges, and let $E_C \subseteq E$ denote the cut-set of C.

- We say that the cut C respects E' if $E' \cap E_C = \emptyset$.
  - **Intuition:** None of the edges in E' crosses the cut $C = (V_1, V_2)$.
- An edge $e \in E_C$ is called a light edge of (G, w) w.r.t. C if its weight is minimized over E_C, i.e. $w(e) = \min_{e' \in E_C} w(e')$.
  - **Intuition:** A light edge has the smallest possible weight among the edges in the cut-set. There could be multiple light edges.

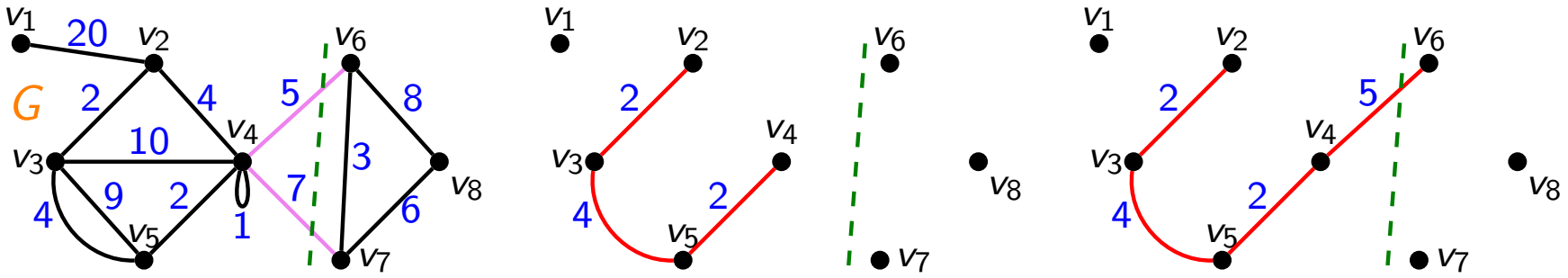
Intuition for computing minimum spanning trees

Let $G = (V, E, \phi)$ be a **connected** undirected graph with n vertices, and let (G, w) be a weighted graph with weight function $w : E \to \mathbb{R}$.

Theorem: Let $C = (V_1, V_2)$ be a cut of G, and suppose $F = (V, E', \phi')$ is a subgraph of G that satisfies our **two invariants** from Slide 18, such that C respects E'. Then any light edge of (G, w) w.r.t. C is a safe edge for F (relative to (G, w)).

- **Intuition:** For any cut C such that no edge of our forest F crosses C, we can always choose any edge in the cut-set (i.e. edges that cross C) with minimized weight to insert into our forest F.

Example: "Before" and "after" for the forest F of a graph G.



Cut $C = (\{v_1, v_2, v_3, v_4, v_5\}, \{v_6, v_7, v_8\})$; edges in cut-set are indicated in purple.

- Among the purple edges in the cut-set, the minimum weight is 5, so we insert into F a new edge (in the cut-set) with weight 5.

Summary

- Trees, forests, rooted trees, DAGs
- Spanning trees
- Vertex labeling, edge labeling, weighted graphs
- Minimum spanning trees, cuts, cut-sets
- Intuition for computing minimum spanning trees